LEARNING & INTERVIEW PLAN · SECOND EDITION

PostCadence
From Eagle's View to Interview Answer

A 14-day plan to understand your own product end to end, turn it into one publishable technical article, and build the vocabulary — and the incident stories — that make a GenAI / backend-automation interviewer believe you actually built it, because you did.

14	2 hrs	28 hrs	4
DAYS	PER DAY	TOTAL	DELIVERABLES

Prepared for Manikanta Sarma Ganti · **Repo** `sud018/linkedin-agent` · **Version** `1.1.0` · **Target** GenAI roles — backend + automation

What changed since the first edition

The app moved on while you were reading the first version of this plan, so this edition is re-read against the repository as it stands today. Two things are different: **Part 1 has a new day** for a subsystem that did not exist before, and **Part 3 has two new days** built entirely out of the problems you hit while running the thing — which is the material interviewers actually dig into.

What is new in the app	Where it lands in this plan
The punctuality subsystem — `agent/cronjob/`, `agent/github/dispatch.py`, `web/routers/trigger.py`, `schedule.py::trigger_times()`	**New Day 5.** An external service now pushes GitHub into running on time, and the app owns that service's jobs. This is the most interview-worthy flow in the codebase.
Automatic four-place sync — saving the schedule writes `config.json`, regenerates `post.yml`, pushes both to GitHub in one commit, and reconciles cron-job.org	Day 5, and a new vocabulary row for *idempotent reconciler*.
Remote drafts in the app — `preview.py::open_drafts()`, `set_draft_text()`, `/drafts/waiting/{n}/…`	Day 2's frontend trace, and Day 4's human-in-the-loop flow.
The history archive — `web/routers/history.py`, `RECENT_ON_DASHBOARD = 5`	Day 3.
Production hardening — `DataFileError`, `SecretsUnavailable`, the global exception handler, `put_files()`, merge-on-pull, cache-busted static URLs, `POSTCADENCE_HOME`	Day 7's data layer, and most of the **new Day 12**.
LinkedIn Little Text Format escaping — `poster.py::escape()`	Day 6, and the best single bug story you own (Day 12).

> **Numbers that moved.** The test suite is now **466 tests**, not 304. The app mounts **11 routers**, not nine. `health.py` runs **7 checks**, not six. If you quote a number in an interview, quote one of these — and re-count before the interview, because they will keep moving.

How to use this plan

Three parts, run in order. Each builds on the one before it — you cannot write the article until you can see the architecture, and you cannot speak fluently in an interview until you have written the article.

Part	Days	What you do	What you end up holding
1 · Architecture	1–7	Trace every flow at the level of "which file, which function, in what order" — no logic, no line-by-line reading.	A one-page system diagram you drew from memory, plus a flow map of every user action.
2 · The article	8–10	Write one technical blog post on an AI engineering topic, using PostCadence as the running example.	A 1,200–1,600 word published post with one diagram. Your portfolio piece and your interview script.
3 · Interview language	11–14	Attach the right vocabulary to things you already built, turn every real failure into an answer, then rehearse until it is automatic.	~70 keywords you can use correctly, an incident log of 20 real problems, 13 rehearsed "what went wrong" answers, and a 90-second project pitch.

The rule that makes this work. Every single day, the last 10 minutes are spent writing 3–5 sentences in a file called `notes/dayNN.md`, from memory, without looking at the code. If you cannot write it from memory, you did not learn it — repeat that day. This is the only quality check in the plan, and it is not optional.

The 2-hour shape of a day

Minutes	Block	What happens
0–15	Recall	Redraw yesterday's diagram from memory on paper. Then check it. The gaps you find are today's real lesson.
15–75	Trace	The day's main work — follow the flow through the files, in order, reading only signatures and calls.
75–110	Produce	Draw, write, or narrate the day's deliverable. Producing is what moves it into memory.
110–120	Notes	Write `notes/dayNN.md` from memory. Closed laptop, closed repo.

Read signatures, not bodies. For Part 1 you never open a function body. In VS Code press `Ctrl+Shift+O` to list every symbol in a file, and `Alt+F12` to peek at a definition without leaving the page. That is the whole toolkit. If you find yourself reading an `if` statement, you have drifted out of eagle's view — close the file and move on.

Every file path, function name and number in this plan was read directly from your repository on **2 October 2026**, at version `1.1.0` — none of it is invented. If a name here does not match your code, your code has moved on, and the difference is worth investigating.

PART 1 · DAYS 1–7

The architecture, from 30,000 feet

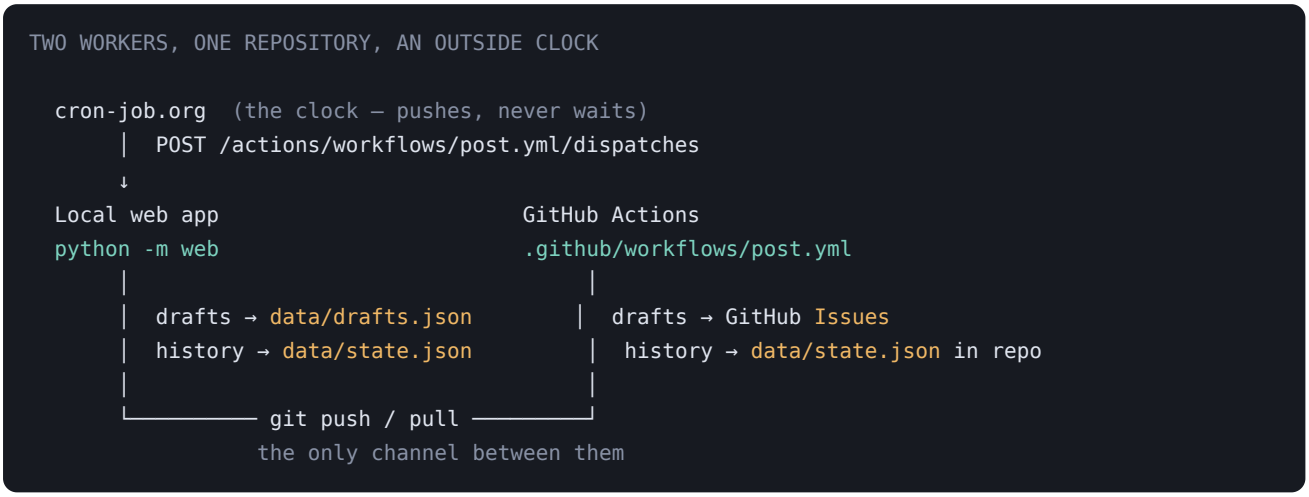
Goal: explain PostCadence to another engineer in five minutes, on a whiteboard, with no notes — naming the files a request passes through and the order it passes through them. Nothing about what is inside a function.

DAY 1 The shape of the whole thing: four layers, two workers, one outside clock

2 hrs

THE ONE IDEA

PostCadence is not one program. It is **two independent workers that share files through git** — a local FastAPI web app you drive by hand, and a GitHub Actions job that runs with your laptop closed. Neither knows the other exists. And because GitHub's own scheduler turned out to be unreliable, there is now a **third party that owns the clock**: an external cron service that pushes GitHub into running. Everything confusing about the product comes from those two facts, so they go in first.



THEN THE FOUR LAYERS

Every one of the 70 Python files in the repo belongs to exactly one of these. Learn the four buckets today and you will always know roughly where a piece of code lives before you go looking for it.

Layer	Lives in	Its one job
Interface	`web/`, `agent/cli/`, `.github/workflows/`	Receive an instruction from a human or a clock. Decide nothing.
Decision	`agent/schedule.py`, `agent/run.py`, `agent/topics/picker.py`, `agent/cronjob/sync.py`	Work out *what should happen* — which slot is due, which topic is next, which cron jobs should exist, post or skip.
Capability	`agent/llm/`, `agent/linkedin/`, `agent/github/`, `agent/cronjob/api.py`, `writer.py`, `formatter.py`	Actually talk to the outside world. Knows how, never why.
Data	`agent/config.py`, `state.py`, `topics/store.py`, `drafts.py`, `jsonio.py`, `paths.py`	Read and write the four JSON files. Nothing else.

> **Notice the split inside `agent/cronjob/`.** `api.py` is capability — it knows how to speak the cron service's REST protocol. `sync.py` is decision — it works out which jobs *should* exist and what to create, update or delete to get there. Two files, two layers, one folder. Being able to point at that and say why it is split that way is a strong answer to "how do you structure code?"

DO THIS

1. Run `find agent web -name "*.py" | sort`. Print the list — it is 70 files.
2. Write one of the four layer names beside every single file. Where you hesitate, that is a file worth a second look later.

3. Draw the diagram above on paper, from memory, three times.

> **Deliverable:** `notes/day01.md` — the two-workers-and-a-clock sentence in your own words, plus your annotated file list.

DAY 2 Frontend flow: how a click becomes a page

2 hrs

THE ONE IDEA

There is **no frontend framework**. No React, no build step, no JSON API for the UI. The server renders complete HTML with Jinja, the browser posts a form, the server redirects, the browser renders again. The nine files in `web/static/js/` only handle the few things a form cannot: polling, live checks, swapping in fragments. Knowing this — and being able to say *why* it was the right call for a single-user local tool — is itself an interview answer.

```
CLICKING "WRITE A DRAFT"

  templates/pages/drafts.html   form POST → /drafts/new
        ↓
  web/routers/drafts.py         new()
        ↓
  agent/run.py                  compose(cfg, slot)
        ├─ agent/topics/store.py   load_topics()
        ├─ agent/topics/picker.py  pick_topic()      → Pick
        ├─ agent/writer.py         write_post()      → calls the model
        └─ agent/formatter.py      format_post()     → plain text, hashtags
        ↓
  agent/drafts.py               add()  → data/drafts.json
        ↓
  redirect 303 → GET /drafts
        ↓
  web/routers/drafts.py         page()
        ↓
  pages/drafts.html + partials/draft_card.html + partials/waiting_card.html
```

THE WIRING ABOVE IT

`web/app.py` → `create_app()` mounts `/static` and includes **eleven** routers. Each owns one URL prefix: `dashboard` (`/`), `history`, `topics`, `wizard` (`/setup`), `drafts`, `github`, `linkedin`, `schedule`, `settings`, `sync`, `trigger`. One router per page is the entire navigation map.

THE SECOND KIND OF DRAFT

The drafts page now shows two different things that look similar and are not. A **local draft** lives in `data/drafts.json` and the app owns it. A **waiting draft** is a GitHub Issue that the scheduled worker opened, and the app is only a remote control for it: `preview.py::open_drafts()` fetches them, and `/drafts/waiting/{n}/approve` posts a `/approve` comment rather than publishing anything itself. Why? Because the Actions run is the only thing holding the LinkedIn token in that flow, and two writers both publishing the same slot is exactly the bug you want to design out.

DO THIS

1. Open `web/app.py` and list the eleven routers with the URL each one owns.
2. Trace three more actions the same way: **Approve & post now** (`POST /drafts/{id}/post` → `drafts.py::publish()` → `run.py::publish()`), **Approve a waiting draft** (`POST /drafts/waiting/{n}/approve` → `_remote()` → `preview.py::say()`), and **Add a topic** (`POST /topics/add` → `topics.py::add()`).
3. Open `web/static/js/trigger.js` and write one sentence on why that page needed JavaScript at all.

> **Deliverable:** four flow diagrams in the style above, drawn by you, and one sentence on local vs waiting drafts.

DAY 3 Backend flow: how the dashboard assembles itself

2 hrs

THE ONE IDEA

The dashboard looks like the most complicated page in the app and is in fact the simplest, because of one design decision: **every loader a page needs is gathered once, in one place** — `web/deps.py::snapshot()`. The route does almost nothing. This is the pattern worth naming out loud in an interview: a single read-model assembled per request, so the template never reaches back into storage.

```
GET /  →  THE DASHBOARD

  web/routers/dashboard.py   dashboard(request)
        ↓
  web/deps.py                snapshot()  → one Snapshot dataclass
        ├─ agent/config.py      load_config()
        ├─ agent/state.py       load_state()
        ├─ agent/topics/        load_topics()
        ├─ agent/schedule.py    local_now()
        ├─ agent/status.py      next_run() streak() counts() recent()
        ├─ agent/analytics.py   daily() by_slot() summary()
        └─ web/charts.py        activity() slot_bars()  → chart geometry
        ↓
  web/health.py              issues()  → what will break tomorrow
        ↓
  pages/dashboard.html + partials/charts.html + partials/issues.html
                                       + partials/run_rows.html  → newest 5 only
```

WORTH NOTICING

Charts are computed **server-side**: `web/charts.py` returns geometry — bar heights, positions, labels — and the template draws SVG. No charting library ships to the browser.

The dashboard shows only the newest five runs (`deps.py::RECENT_ON_DASHBOARD`) with a link to `/history`, which renders the same `partials/run_rows.html` fragment over the full archive with Everything / Posted / Skipped / Failed filters. One fragment, two pages — a small thing, but it is the reason the two views can never disagree about how a run is displayed.

THE SEVEN HEALTH CHECKS

`web/health.py::issues()` is the piece of product thinking in this app. It runs seven checks and sorts them by severity:

Check	What it catches
`check_model()`	No API key for the configured provider — the one failure that stops everything.
`check_linkedin()`	Token missing, or expiring soon. Warns *before* it lapses.
`check_schedule()`	No slots configured at all.
`check_topics()`	Running low, so you find out before the queue is empty.
`check_last_run()`	A slot passed and nothing posted. Carries `dismiss_date` / `dismiss_slot` so you can skip a warning you have already dealt with.
`check_trigger()`	The cron service's jobs no longer match your schedule. Cached for 10 minutes, because this one makes a network call on every page render.
`check_github()`	Not connected, so nothing can run while the laptop is closed.

Say *"proactive health checks"* in an interview and you will be asked to explain it. This table is the explanation. The `dismissable` flag and the 10-minute cache are the details that make it sound like something you ran rather than something you read about.

> **Deliverable:** the snapshot diagram, plus the seven checks listed from memory.

DAY 4 The scheduled worker: how a tick becomes a post

2 hrs

THE ONE IDEA

This is the flow that has no human in it, and the one you will be asked about most. **One run does one thing.** A run that writes the draft does not publish it — it leaves the slot for the next run. Everything happens inside a run; between runs the agent does not exist and no clock is ticking anywhere.

```
A TICK  →  A PUBLISHED POST

  cron-job.org  →  workflow_dispatch  →  .github/workflows/post.yml
        ↓  python -m agent run-due      (mode=due)  or  run --slot HH:MM
  agent/cli/posting.py      cmd_run_due()
        │
        ├─ PHASE 1 — write the draft
        │   agent/schedule.py   prepare_slots()  → what needs drafting
        │   agent/run.py       prepare_preview()
        │        └─ compose() → agent/preview.py create_draft()
        │                  → opens a GitHub Issue, assigns it to you, @-mentions you
        │
        └─ PHASE 2 — decide on an OLDER draft
            agent/schedule.py   due_slots()     → what is postable
            agent/run.py       decide_preview()
                 ├─ preview.py  open_draft()   → finds the issue
                 ├─ preview.py  decide()       → reads /approve /cancel
                 ├─ preview.py  Draft.minutes_open() → review clock
                 ├─ run.py      publish()      → LinkedIn + state
                 └─ preview.py  close_draft()
        ↓
  workflow commits data/state.json back to the repo
```

THE DECISIONS INSIDE `SCHEDULE.PY`

Function	The question it answers
`occurrences()`	"18:30" is a clock time — which *dates* could it mean? Today *and yesterday*, so an evening post survives midnight.
`due_slots()`	Which date+slot pairs are past their time, still inside `catch_up_hours`, and not already handled?
`prepare_slots()`	Which slots need a draft written now, far enough ahead for the review window to fit?
`cron_lines()`	Your local posting times → UTC cron lines, emitting both DST offsets because cron has no idea what daylight saving is.
`trigger_times()`	New. The *local* HH:MM moments the outside clock must fire — slot − preview, slot, and slot + preview when preview mode is on. Three ticks per slot, not one.

Say this in an interview. "GitHub's scheduler is best-effort — I measured delays from 80 minutes to over 6 hours. So I stopped treating the trigger time as the truth. The review window is measured from when the draft was created, not from the wall clock, and a catch-up window bounds how late a post may still go out. Then I went further and took the clock away from GitHub entirely." That one answer demonstrates measurement, root-cause analysis, a design response, and then a second design response when the first was not enough.

Deliverable: the tick diagram from memory, and a written answer to "why does one run never publish what it just

> wrote?"

DAY 5 Punctuality: the outside clock and the four-place sync

2 hrs

WHY THIS DAY EXISTS

GitHub's scheduled runs arrived **5 to 6 hours late**, measured off the run logs. Nothing in the agent could fix that, because the lateness happened before the agent existed. The fix was to notice that GitHub treats two kinds of trigger completely differently: a `schedule:` run goes into a best-effort queue, while a `workflow_dispatch` call is **delivered in seconds**. So the schedule moved outside: an external cron service calls the dispatch endpoint, and `post.yml`'s own cron is now only a fallback.

```
PULL (unreliable)  vs  PUSH (seconds)

  BEFORE   GitHub's own queue  ──wait 5-6 h──→  post.yml runs
           nothing you control is in this line

  AFTER    cron-job.org  POST  /repos/{owner}/{repo}/actions/
                               workflows/post.yml/dispatches
                     │  Bearer: fine-grained PAT, Actions: read+write only
                     │  body: {"ref":"main","inputs":{"mode":"due",...}}
                     ↓
                run queued immediately
```

THE THREE NEW FILES

File	Its one job
`agent/github/dispatch.py`	Capability. `url()`, `body()`, `run_now()` — how to ask GitHub to start a run *right now*. Its docstring records why `workflow_dispatch` was chosen over `repository_dispatch`: the token needs only `Actions: read and write`, where the alternative would have needed write access to your repository contents.
`agent/cronjob/api.py`	Capability. The cron service's REST protocol: `list_jobs()`, `create_job()`, `update_job()`, `delete_job()`, `get_job()`, `whoami()`, and a `CronJobBusyError` raised on HTTP 429.
`agent/cronjob/sync.py`	Decision. `build()` describes the job a slot *should* have; `stale()` compares what exists against that; `plan()` works out the creates, updates and deletes; `apply()` performs them; `drift()` reports without changing anything.

THE PATTERN TO LEARN BY NAME: AN IDEMPOTENT RECONCILER

This is the most transferable idea in your codebase, because it is how Kubernetes, Terraform and every infrastructure tool works. `sync.py` never says "add a job". It says: *here is the set of jobs that should exist; here is the set that does; converge them.* Run it once or fifty times and the end state is identical. Ownership is marked by a title prefix — `PREFIX = "PostCadence · "` — so the reconciler will never touch a job you created by hand.

```
RECONCILE

  wanted  = schedule.py trigger_times(cfg)     → {"08:30","09:00","09:30"}
  current = cronjob/api.py list_jobs()         → filtered by PREFIX
        ↓
  cronjob/sync.py plan()
        ├─ in wanted, not in current   → create_job()
        ├─ in both but stale()         → update_job()
        └─ in current, not in wanted   → delete_job()
        ↓
  end state is the same however many times you run it
```

THE FOUR-PLACE SAVE

A posting time used to live in four places that could silently disagree: your config, the workflow file, GitHub's copy of both, and the cron service. Now `web/routers/schedule.py::save()` does all four in order, and reports each failure separately without undoing the save:

```
POST /schedule  →  SAVE

  web/routers/schedule.py  save()
        ├─ 1  agent/config.py        save_config()      → data/config.json
        ├─ 2  web/workflow.py        regenerate cron      → .github/workflows/post.yml
        ├─ 3  _push_to_github()      github/api.py put_files()
        │                               → BOTH files in ONE commit
        └─ 4  _push_to_service()     cronjob/sync.py apply()
```

Why one commit matters. Pushing the two files as two commits left an intermediate commit where `config.json` and `post.yml` disagreed — and CI runs on every commit, so `tests/test_workflow_matches_config.py` went red every single time you saved your schedule. `api.py::put_files()` builds one commit for many files through GitHub's Git Data API: read the ref, read the commit, build a tree, create a commit, move the ref. Five calls, one commit, no inconsistent state ever visible. That is a complete interview story on its own and it lives on Day 12.

DO THIS

1. Trace `POST /setup/trigger/sync` from `web/routers/trigger.py` through to `api.py::update_job()`, naming each function in order.
2. Read `sync.py::stale()`'s signature and its docstring only. Then answer: why does it compare only the fields the *listing* returns, rather than fetching each job's full detail? (The answer is on Day 12, and it is HTTP 429.)
3. Say out loud, three times: "I replaced a pull-based best-effort schedule with a push-based one, and made the app the reconciler for it."

Deliverable: the reconcile diagram and the four-place save diagram, drawn from memory, plus one paragraph defining *idempotent reconciler* in your own words.

DAY 6 Capability layer: the model, LinkedIn, GitHub, the cron service, the secrets

2 hrs

THE ONE IDEA

Every outside system is reached through a **swappable adapter**, and no secret is ever written to a file in the repo. Two sentences, and they carry more interview weight than any amount of prompt talk.

Capability	Entry point	Shape
Model	`agent/llm/__init__.py` → `get_provider()`, `build()`	Abstract base `LLMProvider` in `base.py`; four implementations — openai, anthropic, gemini, ollama. Swapping provider changes one config field.
Writing	`agent/writer.py` → `build_prompt()`, `write_post()`	Prompt assembled from tone, author context, the picked topic and recent topics — then handed to whichever provider is configured.
Cleanup	`agent/formatter.py` → `format_post()`	Deterministic, no model: strips markdown, normalises bullets, adds up to 3 hashtags, enforces the 3000-character limit at a sentence boundary.
LinkedIn	`agent/linkedin/oauth.py`, `poster.py`, `diagnose.py`	OAuth authorization-code flow: `authorize_url()` → callback → `finish()` → `exchange_code()` + `fetch_member_id()`. Posting is `post_text()`, which now escapes LinkedIn's markup before sending.
GitHub	`agent/github/api.py`, `device.py`, `dispatch.py`, `repo_secrets.py`	Device flow for sign-in (no client secret on the machine); `put_files()` for one-commit writes; `dispatch.py` to start a run now; `repo_secrets.py` seals secrets with libsodium before sending them to Actions.
Cron service	`agent/cronjob/api.py`	Bearer-auth REST. Raises `CronJobBusyError` on 429 so callers can back off instead of failing. Error messages carry the status code only — never the response body, which could contain the key.
Secrets	`agent/secrets_store.py` → `get_secret()`	Environment variable first, then the OS keyring (Windows Credential Manager). Raises `SecretsUnavailable` rather than crashing when the keyring itself is broken. Never a file, never the repo.

THE SEPARATION THAT MATTERS MOST

`writer.py` is the *only* file that calls a model. `formatter.py` runs right after it and is completely deterministic. That boundary — **the model proposes, ordinary code disposes** — is the single most valuable thing in this codebase to be able to articulate. Day 8's article is built on it.

AND ONE MORE BOUNDARY, ADDED LAST

`poster.py::escape()` sits between the formatter's output and LinkedIn's API, because `commentary` is not a plain-text field — LinkedIn parses it as "Little Text Format", in which `| { } @ [ ] ( ) < > # \ * _ ~` are all markup and must arrive backslash-escaped. Note what the function deliberately does *not* escape: a real hashtag's `#`, because escaping it would publish `\#RAG` as dead text instead of a tag. **One layer's output is another layer's input, and the two can disagree about what the characters mean.** Learn that sentence; it is the general lesson behind SQL injection, XSS, CSV injection and this bug, all at once.

Deliverable: one paragraph on why `LLMProvider` is an abstract base class instead of four `if` statements, and one on why escaping belongs in `poster.py` rather than in `formatter.py`.

DAY 7 Data layer, then assemble the whole picture

2 hrs

FOUR FILES, AND WHO OWNS EACH

File	Module	Direction
`data/config.json`	`agent/config.py`	Flows **up** — laptop is the source of truth, pushed to the repo.
`data/topics.json`	`agent/topics/store.py`	Flows **up**.
`data/state.json`	`agent/state.py`	Flows **down** — Actions owns it. `push_state()` is the deliberate exception, and it merges rather than overwrites.
`data/drafts.json`	`agent/drafts.py`	Local only, gitignored. GitHub's equivalent is an Issue.

FOUR THINGS IN THIS LAYER THAT EACH EXIST BECAUSE OF A REAL FAILURE

What it does	Why it is there
`jsonio.py::write_atomic()`	Writes to a temp file, `flush()`, `fsync()`, then `os.replace()`. Without the fsync the rename can reach the disk before the bytes do, and the file comes back empty.
`jsonio.py::_replace()`	Retries `PermissionError` five times with backoff, because on Windows an anti-virus scanner or the search indexer can hold the destination file open for a moment.
`jsonio.py::DataFileError`	A hand-edited or half-written JSON file used to raise a bare `JSONDecodeError` from deep inside a loader. Now the error names the file and the line.
`github/sync.py::merge()`	Keys history entries on `(date, slot, created_at)`, so syncing the same entry back and forth is a no-op. `pull_state()` merges — it used to replace, which would have deleted local history.

One more: `agent/paths.py` reads `POSTCADENCE_HOME` before falling back to the repo directory. That single line is what lets the test suite point the whole app at a sandbox — and the reason it exists is a test run that overwrote the real `post.yml`. Also on Day 12.

THE ASSEMBLY — 60 MINUTES, NO REPO OPEN

1. One blank page. Draw the entire system: the outside clock, two workers, four layers, four data files, four external services.
2. On the same page, mark the path of a scheduled post in one colour and a manual post in another.
3. Record yourself explaining it out loud for five minutes. Watch it back. Every place you said "um, and then it kind of…" is a gap.
4. Only now open the repo and fix what you got wrong.

Deliverable: `docs/architecture.md` with your diagram, committed to the repo. Part 1 is complete when you can draw it twice in a row without checking.

PART 2 · DAYS 8–10

One article that does the talking for you

A short, specific technical post — 1,200–1,600 words — on an AI engineering topic, with PostCadence as the running example. Written once, it becomes your portfolio piece, your LinkedIn post, and the script you fall back on when an interviewer says "tell me about something you built."

DAY 8 Choose the angle and outline it

2 hrs

PICK ONE OF THESE SIX

Title	The argument	Why it works
★ **The model writes. Ordinary code decides.**	An LLM should produce content, never control flow. Everything about *when*, *whether* and *how often* belongs in deterministic code.	**Recommended.** It is the real lesson of your codebase, it is a genuine industry debate, and `writer.py` vs `formatter.py` vs `run.py` is the perfect three-part example.
Your agent does not own its clock	Scheduled triggers are best-effort. Design for lateness as the normal case — then take the clock away from the platform entirely.	Now much stronger than it was: you have measured data *and* a push-based fix with a reconciler behind it.
The approval window	How to put a human in the loop without making them a bottleneck — timeouts, fail-open vs fail-closed.	Very current. Every company shipping AI is arguing about this.
Two workers, no database	Reconciling state between a laptop and a CI runner using git and an idempotent merge.	Reads as strong distributed-systems thinking from a small project.
Where do the keys live?	Credential handling for a personal agent: keyring, OS store, sealed CI secrets, OAuth scopes, least-privilege PATs.	Security-flavoured and concrete. Safe, slightly narrower.
The post that published half of itself	Every boundary between two systems is a chance for them to disagree about what a character means. One bug, told properly, from symptom to the general rule.	Short, vivid, and the single most memorable thing you could write. Great second article.

OUTLINE TO FILL IN (USE THE ★ ANGLE)

1. **The hook** — 28 September: the agent wrote a draft and published nothing. Open with the real log lines.
2. **What people usually build** — the "let the model decide" agent, and why it is appealing.
3. **The split** — model proposes, code disposes. Name your three files.
4. **What went in the deterministic half** — slot selection, the due/catch-up rules, idempotency, the formatter, the escaper, the failure record.
5. **What the model actually got** — one job: turn a topic into prose in a given voice.
6. **The payoff** — a bug in scheduling is reproducible and testable; 466 tests, none of which need a model.
7. **The honest limit** — what this split costs you, and when you would choose differently.

> **Do not skip point 7.** An article that admits a tradeoff reads as engineering. An article that does not reads as marketing — and an interviewer will probe exactly there.

> **Deliverable:** chosen title, seven headings, and two sentences under each.

DAY 9 Write the draft

2 hrs

Ninety minutes, straight through, no editing. Rules: one concrete example per section; real numbers only (5–6 hours late, 466 tests, 70 Python files, 4 JSON files, 7 health checks); real file names; no sentence you could not defend if asked "why?" three times. Last thirty minutes: read it aloud and cut every sentence that does not carry a fact.

THE AI TOPICS THIS ARTICLE LETS YOU TOUCH

This is the list you asked for — concepts a GenAI interviewer expects, each one already present in your app, so you can speak about it from experience rather than theory.

Concept	Where it already lives in PostCadence
Agent vs chatbot vs workflow	Your app is an *agentic workflow*: a fixed pipeline with one model call, not an autonomous loop. Knowing the difference is half the answer.
Prompt construction & context injection	`writer.py::build_prompt()` — tone, author context, topic, recent topics to avoid repetition.
Output contract & post-processing	`formatter.py` — the model's output is never trusted raw.
Output encoding at a trust boundary	`poster.py::escape()` — the model's text is also never trusted as *markup*.
Determinism boundary	The whole article.
Human-in-the-loop / approval gates	Preview mode, the GitHub Issue, `/approve` and `/cancel`, and the app's own remote control over them.
Fail-open vs fail-closed	`preview_timeout_action`: silence means post, or silence means skip. A real policy choice.
Idempotency & exactly-once	A slot is closed once; `already_handled()`; the merge key in `github/sync.py`.
Reconciliation / desired-state convergence	`cronjob/sync.py` — wanted set vs current set, converge, ownership by prefix.
Push vs pull scheduling	`dispatch.py` and `trigger_times()` — why a dispatch arrives in seconds and a schedule does not.
State reconciliation	Two writers, one `state.json`, no database.
Observability of failures	`record_failure()` + `health.py`'s seven checks — a failure that leaves no trace looks exactly like a quiet day.
Rate limits & backoff	`CronJobBusyError` on 429, plus the 10-minute health-check cache that stopped causing them.
Provider abstraction & cost	`LLMProvider`; one call per post, so cost is trivially bounded.
Secrets & least privilege	`secrets_store.py`, sealed Actions secrets, the `w_member_social` scope, a PAT scoped to `Actions` alone.
Evaluation & regression testing	466 tests, none requiring a live model — because the model was kept out of the decisions.

Deliverable: full messy draft. Do not show it to anyone yet.

DAY 10 **Edit, diagram, publish** 2 hrs

- **0–40 · Cut.** Target 1,400 words. Delete every adjective that is not load-bearing. Replace every "significantly improved" with a number.
- **40–70 · One diagram.** Exactly one. The three-way split: `picker` / `run` (decides) → `writer` (generates) → `formatter` / `poster` (validates and encodes). Mark clearly which single box contains the model.
- **70–95 · The defence pass.** For each claim, write the hardest question an interviewer could ask, and answer it in one line. Keep these answers — they feed Day 13.
- **95–120 · Publish.** LinkedIn article or dev.to, plus `docs/` in the repo. Link the repo. Then post it from the app itself, and say so in the post — that is the closing line that people remember.

Deliverable: a published URL, and a list of the hard questions with your answers.

PART 3 · DAYS 11–14

The language, and the war stories

Keywords only help when they are attached to something you actually did — so Day 11 pairs every term with the exact place it lives in your app. Then Days 12 and 13 do the thing that actually wins interviews: turn each real failure you hit into an answer with a symptom, a root cause, a fix, and a lesson. You have twenty of them. Most candidates have none.

DAY 11 Vocabulary, anchored to your own code

2 hrs

Spend the two hours saying each phrase out loud, immediately followed by its anchor. Never learn the word alone.

Agent & system design

Phrase	Your anchor
Agentic workflow	"Mine is a deterministic pipeline with one model call — not an autonomous loop. I chose that deliberately."
Deterministic scaffolding	The code around the model: `schedule.py`, `run.py`, `formatter.py`.
Control plane / data plane	Decision layer vs capability layer — and `cronjob/sync.py` vs `cronjob/api.py` in the same folder.
Provider abstraction	`LLMProvider` ABC, four backends, swapped by config.
Separation of concerns	The four layers.
Orchestration	`cmd_run_due()` — two phases, one run.
Self-hosted / no vendor lock-in	"Users run their own copy. That shaped every integration choice — anything I depend on, they have to be able to create an account for themselves, and the setup guide says exactly how."

Working with the model

Phrase	Your anchor
Prompt construction	`build_prompt()`.
Context injection	Recent topics passed in so the model avoids repeating itself.
Output contract	Plain text, length-bounded, no markdown — enforced by code, not by asking nicely.
Post-processing / validation layer	`format_post()`.
Output encoding	`poster.py::escape()` — the step after validation, where text becomes someone else's markup.
Grounding	The topic list — the model never invents what to write about.
Non-determinism	"Two runs on the same topic give different wordings. That is a feature in the Rewrite button and a hazard everywhere else."
Token & cost bounding	One call per post; cost is a function of posts, not of user behaviour.

Reliability — the strongest section for backend + automation roles

Phrase	Your anchor
Idempotency	A slot closes once; `already_handled()` guards re-entry.
Idempotent reconciler	`cronjob/sync.py` — wanted set vs current set, converge; ownership by title prefix.

Phrase	Your anchor
Desired-state configuration	"Same model as Terraform or a Kubernetes controller, at a much smaller scale."
Push vs pull trigger	`workflow_dispatch` arrives in seconds; a `schedule:` run joins a best-effort queue.
At-least-once delivery	Cron may fire late or twice; the code must tolerate both.
Catch-up window	`catch_up_hours` — bounded lateness, so nothing posts at 3am.
Reconciliation / idempotent merge	`github/sync.py::merge()`, keyed on `(date, slot, created_at)`.
Atomic write	`jsonio.py` — temp file, fsync, rename, with a retry for Windows file locks.
Rate limiting & backoff	`CronJobBusyError` on HTTP 429, and the cache that stopped us causing them.
Caching with a TTL	`health.py`'s 10-minute `DRIFT_TTL` — a network call that was happening on every page render.
Failure surfacing	`record_failure()`, deduplicated so four ticks do not mean four identical entries.
Graceful degradation	A failure never closes a slot, so a later run retries. A broken keyring shows a setup page, not a 500.
Drift detection	`health.py::check_trigger()` compares your configured schedule against the jobs that actually exist.
Atomic multi-file commit	`github/api.py::put_files()` — because two commits meant an inconsistent intermediate state that CI could see.

Human oversight

Phrase	Your anchor
Human-in-the-loop	Preview mode.
Approval gate	The GitHub Issue with `/approve` and `/cancel`.
Fail-open vs fail-closed	`preview_timeout_action` — a configurable policy, not a hardcoded assumption.
Audit trail	`state.json` history, committed to the repo by the workflow.
Review SLA	The window measured from draft creation, not wall-clock.
Single writer per resource	The app approves by commenting; only the Actions run publishes. Two publishers for one slot is a bug you designed out.

Security & operations

Phrase	Your anchor
Secrets at rest	OS keyring, never a repo file.
Sealed secrets	libsodium sealed boxes via PyNaCl before sending to Actions.
OAuth authorization-code flow	LinkedIn, with exact redirect-URI matching.
Device authorization flow	GitHub sign-in with no client secret on the machine, including the `slow_down` backoff the spec requires.

Phrase	Your anchor
Least-privilege scopes	`w_member_social`; a fine-grained PAT with `Actions: read and write` and nothing else; `introspect()` verifies what was actually granted.
Command injection	Interpolating an input straight into a shell line in a workflow. You had one; you fixed it by passing through the environment and validating the format.
Secret leakage through error messages	A token-exchange error that quoted the response body ended up urlencoded into a redirect URL. Now the message carries the status code only.
Token lifecycle	Expiry tracked in config; the dashboard warns before it lapses.

> **Deliverable:** every phrase said aloud with its anchor, twice, without reading.

DAY 12 · THE SESSION YOU ASKED FOR

The incident bank: twenty problems you actually solved

"What went wrong and how did you handle it?" is the question that separates people who built something from people who watched a tutorial. You cannot invent a good answer to it — but you do not need to, because you have twenty real ones. This day is about writing them down in a form you can say out loud.

How to use this day. Do not try to memorise twenty stories. Read all twenty once, pick the **five** marked ★, and write those five out fully in the four-line shape on the next page. The other fifteen exist so that when an interviewer asks a follow-up — "anything with security?", "ever broken your own CI?" — you have something true to reach for instead of going blank.

Group A · Time, schedules and things that did not happen

Symptom	Root cause	What you did	What it proves
★ **1. The post that never went out.** 28 Sep: a draft was written, nothing was published, and no error appeared anywhere.	**Three separate causes stacked.** (a) The Actions run arrived 80–110 minutes late. (b) The run that created the draft then judged that same draft, before any human could see it. (c) An evening slot stopped existing at midnight, because a clock time was being matched only against today's date.	Found them in order, one per hypothesis. Split a run into two phases so a run never decides on a draft it just created. Made `occurrences()` return today *and* yesterday. Measured the delay rather than guessing at it.	Layered debugging. Most bugs people describe have one cause; being able to say "there were three, and here is the order I found them in" is the whole answer.
★ **2. Scheduled runs 5–6 hours late.** Measured, repeatedly, from the run logs.	GitHub Actions `schedule:` triggers are best-effort and were platform-wide degraded. Nothing inside the app could affect it — the lateness happened before the app started.	Noticed that `workflow_dispatch` is *delivered*, not queued, and arrives in seconds. Moved the clock outside to a cron service that calls the dispatch endpoint, and left `post.yml`'s own cron as a fallback.	Reading the platform's own semantics instead of accepting its behaviour; and distinguishing pull from push.
3. One posting time, three places to change it — and they silently disagreed.	`config.json`, the cron in `post.yml`, and the external service's jobs were each updated by hand.	Made saving the schedule do all four writes in order, each failure reported separately, none of them undoing the save. Added a health check that detects the drift if it happens anyway.	Removing a manual step is a design decision, not a convenience. And: detect the state you tried to make impossible.
4. A draft could expire before anyone had a chance to look at it.	The review window was measured against the slot's wall-clock time. If the run was two hours late, the window had already closed.	Measured the window from `Draft.minutes_open()` — when the draft was actually created — so lateness delays the post rather than cancelling the review.	Choosing the right clock. A deadline measured against something you do not control is not a deadline.

Group B · Two systems disagreeing about what a value means

Symptom	Root cause	What you did	What it proves
★ **5. LinkedIn published the first half of a sentence.** The draft read "…is retriever-augmented generation (RAG) still necessary?" and the published post stopped dead at "generation".	LinkedIn's `commentary` field is not plain text — it is parsed as "Little Text Format", where `| { } @ [ ] ( ) < > # \ * _ ~` are markup. We sent the text raw, so the parser stopped at the first unescaped `(`. Latent since the poster was written; it only surfaced when a topic happened to contain a bracket.	Read the API documentation rather than guessing. Escaped all fourteen characters, backslash first so nothing is escaped twice — but deliberately left a real hashtag's `#` alone, because `\#RAG` publishes as dead text instead of a tag. Counted the length *before* escaping, since the backslashes are markup, not characters a reader sees. Twenty tests, one per character plus the hashtag round-trip.	The best single bug you own. It demonstrates: reading a spec, a latent bug surfacing from data, a fix with a deliberate exception, and the general rule that **every boundary between two systems is a chance for them to disagree about what a character means** — the same rule behind SQL injection and XSS.
★ **6. "Punctual schedule out of date" would not clear.** Syncing reported "6 jobs updated" every single time, forever.	The cron service's `GET /jobs` listing *omits* the `extendedData` field that holds each job's headers and body. So every job looked like it was missing its configuration, so every job looked stale.	Compared only the fields the listing actually returns, and moved the full rewrite behind the explicit Sync button (`rewrite=True`).	An API's read shape is not its write shape. "I assumed the list endpoint returned the same object as the detail endpoint" is an honest, specific, very relatable answer.
7. GitHub mobile notifications for new drafts never arrived, even with notifications on.	"An issue was opened in a repo you watch" is not one of GitHub Mobile's push types. Direct mentions and assignments are.	Had `create_draft()` assign the issue to the repo owner and @-mention them in the body — two things that *are* push types.	Working within a platform's actual rules instead of arguing with them. Also: checking the documented behaviour before suspecting your own code.
8. Signing in to LinkedIn from the command line failed for anyone who followed the setup guide.	The CLI used a different redirect URI than the one the setup guide tells you to register, and LinkedIn matches that URL character for character. Worst possible timing: that command is what you run when your token has already expired.	One constant, used by both paths. The callback server now binds exactly that host, port and path, with a clear error if the port is taken because the app is already running.	Two code paths that must agree about a value should read it from one place. And the cost of a bug is partly *when* it fires.

Group C · The cost of a check, and being told to slow down

Symptom	Root cause	What you did	What it proves
★ **9. HTTP 429 — rate-limited by the cron service.** Caused by the first attempt at fixing problem 6.	That fix fetched each job's full detail to compare it properly. But the drift check runs inside `health.py`, which runs on *every page render* — so browsing the app made seven API calls a page.	Three changes together: compare using the one listing call instead of seven detail calls; cache the result for 10 minutes (`DRIFT_TTL`); and raise `CronJobBusyError` on 429 so the health check quietly returns nothing instead of showing an error.	The strongest story in the set, because the bug was *in your own fix*. It shows that you measure the cost of a correctness fix, and that you own a regression you caused without being defensive about it.

Symptom	Root cause	What you did	What it proves
10. GitHub device sign-in could be refused indefinitely.	GitHub's device flow answers `slow_down` when you poll too fast, and requires you to add 5 seconds to the interval. We treated that answer as ordinary "still waiting" and kept polling at the same rate — so it could answer `slow_down` forever.	Modelled the response properly as an `Answer` with a `too_fast` flag, and had the browser widen the polling gap when it sees one.	Reading the whole spec, not just the happy path. "The protocol told me what to do and I was not listening" is a good, humble, technically precise answer.

Group D · Breaking your own CI and your own data

Symptom	Root cause	What you did	What it proves
★ **11. CI went red every time you saved your schedule** — even though the final state was correct.	The two files were pushed as two commits. That left an intermediate commit where `config.json` and `post.yml` disagreed — and CI runs on every commit, so `test_workflow_matches_config.py` failed on the one in the middle.	Wrote `github/api.py::put_files()`, which builds a single commit for many files through the Git Data API: read the ref, read the commit, build a tree, create a commit, move the ref. The inconsistent state now never exists.	Atomicity is not only a database idea. And a test that is "flaky" is sometimes telling you the truth about a state you did not know you were creating.
12. Running the test suite rewrote the real `.github/workflows/post.yml`.	`paths.ROOT` pointed at the actual repository, so a test that exercised workflow generation wrote to the live file.	Made `paths.py` read `POSTCADENCE_HOME`, and had `tests/conftest.py` point the whole app at a sandbox with a fake `post.yml`. Then verified the real file was byte-identical after three full runs.	Tests that touch real state are worse than no tests. Also: the verification step — proving the fix, not assuming it.
13. Pulling history from GitHub would have deleted local history. Found by audit, never shipped.	`pull_state()` replaced the local file with the remote one, rather than merging. Any run recorded locally and not yet pushed was gone.	Made it merge, keyed on `(date, slot, created_at)` — the same key `merge()` already used, so syncing the same entry back and forth is a no-op.	Reading your own code looking for data loss specifically, and finding it before a user does. Say "I audited for destructive paths" — then give this example.
14. Saving a file occasionally failed on Windows with `PermissionError`, on a file that was fine a second later.	An anti-virus scanner or the search indexer holds the destination file open for a moment, and `os.replace()` cannot swap it. Separately, without an `fsync()` the rename can reach the disk before the bytes do and the file comes back empty.	`jsonio.py`: `flush()` + `fsync()` before the rename, and `_replace()` retries five times with increasing backoff.	Knowing what an atomic write actually requires, and that "works on my machine" and "works on Windows" are different claims.

Group E · The security pass

Symptom	Root cause	What you did	What it proves
15. A workflow input was interpolated straight into a shell command.	`post.yml` ran `python -m agent run --slot ${{ inputs.slot }}`. GitHub substitutes that *before* the shell sees it, so anyone who could dispatch the workflow could run arbitrary commands on the runner.	Passed the value through the environment instead (`SLOT: ${{ inputs.slot }}`) and validated it against `^[0-2][0-9]:[0-5][0-9]$` before using it.	Command injection recognised in a place most people never look. The textbook fix, correctly applied: never interpolate into a command, validate the shape.
16. A failed LinkedIn token exchange could put the client secret into a URL.	The error message quoted the response body, and the page urlencoded that message into a redirect — so a secret could land in a URL, in browser history, and in any log that records URLs.	The error now carries the status code only, with a comment in the code explaining why the body must never be included.	Thinking about where an error message *travels*, not just what it says.
17. A hand-edited or half-written JSON file crashed the app with a 500 and a stack trace, with no route back to setup.	A bare `JSONDecodeError` raised from deep inside a loader, and nothing caught it.	Added `DataFileError` naming the file and line, `SecretsUnavailable` for a broken keyring, and one global exception handler in `web/app.py` that renders a "needs setup" page instead.	Error messages as a product feature, and failing to a usable state rather than a traceback.

Group F · The browser, where the obvious explanation was wrong

Symptom	Root cause	What you did	What it proves
18. GitHub sign-in spun forever with no error.	The helper that posts to the server threw when the response was a non-JSON 500 — inside an async callback nobody awaited. The exception vanished and the spinner never stopped.	Handled HTTP errors explicitly, wrapped the callback in try/catch, capped the wait at 15 minutes, and added a "Start over" button.	An unhandled rejection in a fire-and-forget callback is invisible. Worth knowing by name.
19. Hiding a button with `element.hidden = true` did nothing.	The stylesheet's `.btn { display:inline-flex }` is more specific than the browser's built-in `[hidden]` rule, so it won.	Added `[hidden] { display:none !important; }`.	CSS specificity, diagnosed rather than worked around by adding another class.
20. A CSS and JavaScript fix had no effect, repeatedly, even though the files were correct on disk.	The browser was serving cached copies.	Added `?v=…` to every stylesheet and script, stamped from the newest static file's modification time, so the URL changes whenever the content does.	Cache-busting, and the discipline of not assuming your fix is wrong when it might not have shipped. Useful, slightly funny, and very human.

TODAY'S WORK

1. **0–30.** Read all twenty. Against each, write one word: the *category* it belongs to (timing, boundary, cost, data, security, browser). Categories are what you will reach for under pressure, not individual stories.
2. **30–100.** Write the five ★ stories out fully, in the four-line shape on the next page. Ten to fifteen minutes each.

3. **100–120.** Say each of the five out loud once, timed. Anything over 90 seconds is too long — cut the setup, never the root cause.

> **Deliverable:** `notes/incidents.md` — all twenty in one line each, and the five ★ written out in full.

DAY 13 · THE SESSION YOU ASKED FOR

Turning an incident into an answer

Having the story is not the same as telling it. An interviewer is listening for four things in a fixed order, and most candidates give them two. Today you learn the shape, map every question they might ask onto one of yesterday's twenty, and then answer out loud until it stops feeling like recitation.

THE FOUR-LINE SHAPE — USE IT FOR EVERY SINGLE ANSWER

Line	What goes in it	Worked example — incident 9
1 · Symptom one sentence, concrete	What you observed, with a number or a quote. Never the cause yet.	"Browsing my own dashboard started getting rate-limited — HTTP 429 from the cron service."
2 · Root cause one sentence, honest	The actual mechanism. If it was your fault, say so plainly — it reads as confidence, not weakness.	"My own fix from the day before. I'd started fetching each job's full detail to compare it properly, and that check runs on every page render — so every page was seven API calls."
3 · The fix two sentences, specific	What you changed, and why that was the right shape of fix rather than the first one you thought of.	"Three things: compare using the single listing call instead of seven detail calls, cache the result for ten minutes, and treat a 429 as 'unknown' rather than 'error' so the health check stays quiet. The cache is the real fix — the check didn't need to be fresh, it needed to be cheap."
4 · The lesson one sentence, general	The transferable rule. This is the line that makes them think you will not repeat it.	"I now ask what a correctness fix costs per call site before I ship it — especially anything that runs on a render path."

Why this order. Symptom first gives the interviewer something to be curious about. Cause second answers that curiosity and proves you found it rather than guessed. Fix third is the engineering. Lesson last is what they actually remember. Say the lesson first and the story sounds rehearsed; leave it out and you sound like you got lucky.

The question map — every "what went wrong" question, and which incident answers it

What they ask	Reach for
"Tell me about the hardest bug you have debugged."	**Incident 1** — three stacked causes, found in order. Lead with "there were actually three things."
"Tell me about a problem you found that was your own fault."	**Incident 9** — the 429 caused by your own fix. Best possible answer to this one.
"How do you debug something you cannot reproduce?"	**Incident 2** — you could not reproduce GitHub being slow, so you measured it instead, run log against cron schedule, repeatedly, until you had a range.
"A time you made an incorrect assumption."	**Incident 6** — you assumed a list endpoint returns the same object as the detail endpoint. Or **8**: you assumed two code paths agreed about a URL.
"Tell me about a bug that only appeared in production."	**Incident 5** — the escaping bug had been there since the file was written; it only fired when a generated topic happened to contain a bracket.

What they ask	Reach for
"Have you ever broken your own build?"	**Incident 11** — CI red on every save, because of an intermediate commit nobody had thought about.
"How do you handle a dependency you do not control?"	**Incident 2**, then the design response: catch-up windows, retries that do not duplicate, and finally taking the clock away from the platform.
"Tell me about a time you nearly lost data."	**Incident 13** — `pull_state()` replacing instead of merging. Found by audit, never shipped.
"Have you found a security issue in your own code?"	**Incident 15** — command injection in the workflow. Then **16** — the secret that could land in a URL.
"How do you know your tests are any good?"	**Incident 12** — the tests were writing to the real repository. A test suite with side effects is not a safety net.
"A time something took far longer than you expected."	**Incident 20** — a fix that was correct on disk and cached in the browser. Honest, mildly funny, and everyone has done it.
"What would you do differently if you started again?"	Escape at the boundary from day one (5); never let a check run on a render path without a cache (9); sandbox `paths.ROOT` before writing the first test (12).
"What is still broken or unfinished?"	Answer honestly: five near-duplicate `_back()` helpers across routers that you consciously deferred; no second platform, so the abstraction is unproven; no evaluation harness scoring generated posts.

The four ways this answer goes wrong. **Blaming the platform and stopping there** — "GitHub was slow" is a symptom, not a story; it only counts once you say what you did about it. **No number** — "it was quite late" is forgettable; "5 to 6 hours, measured off the run logs" is not. **Skipping the root cause** — if you jump from symptom to fix, they will assume you changed things until it worked. **Claiming a tidier story than the truth** — if you did not find it by reading the spec, do not say you did. One follow-up question and the whole interview changes character.

Two answers written out in full — copy the shape, not the words

"Tell me about the hardest bug you have debugged." — incident 1

"My agent wrote a post one evening and published nothing, with no error anywhere. It turned out to be three separate things stacked on top of each other, and I had to peel them off one at a time. First, the scheduled run was arriving about an hour and a half late, which I only knew because I compared the run log timestamps against the cron schedule. Second — and this was mine — the same run that created the draft then judged it, so it was deciding on a draft that no human had had a chance to see; I split a run into two phases so that can't happen. Third, the slot was an evening one, and by the time a late run arrived it was past midnight, so the code was looking for that clock time only on today's date and the slot had effectively ceased to exist. Now a clock time resolves against today *and* yesterday. The lesson I took is that when nothing at all happened, there is usually more than one reason, so I stopped looking for *the* cause and started removing them one at a time."

"Tell me about a problem that was your own fault." — incident 9

"I got rate-limited by my own app. I'd fixed a comparison bug by fetching each scheduled job's full detail from the API, which was correct — but that comparison runs inside my dashboard's health check, and the health check runs on every page render. So just browsing my own app was making seven API calls a page, and the service started returning 429. I fixed it three ways: I went back to comparing only the fields the list endpoint returns, so it's one call; I cached the result for ten minutes; and I made a 429 mean 'I don't know' rather than 'error', so the user never sees a scary warning because of a rate limit. The cache is the actual fix — that check never needed to be fresh, it needed to be cheap. Since then I ask what a correctness fix costs per call site before I ship it, especially on anything that runs on a render path."

TODAY'S WORK

1. **0–20.** Read the question map. For each row, say only the incident *number* out loud, fast, until the mapping is instant. The mapping matters more than the wording.
2. **20–80.** Answer all thirteen questions out loud, in the four-line shape, recording yourself. No notes.
3. **80–110.** Listen back. Mark every answer where you skipped line 2 or line 4 — those are the two people drop. Redo those.
4. **110–120.** Write the two questions that still made you reach. Those are Day 14's homework.

Deliverable: thirteen recorded answers, each under 90 seconds, each with all four lines present.

DAY 14 Rehearsal, the project pitch, and the phrases that land

2 hrs

THE 90-SECOND PITCH — MEMORISE THE SHAPE, NOT THE WORDS

"Tell me about a project."

"I built an agent that writes and publishes my LinkedIn posts on a schedule. The interesting part is not the generation — that is one model call. It is everything around it. It runs on GitHub Actions, so it has no persistent process and no control over its own clock. I measured scheduled runs arriving five to six hours late, so I stopped treating the trigger time as the truth — the review window is measured from when the draft was created, and a catch-up window bounds how late a post may still go out. Then I went further and took the clock off GitHub entirely: an external cron service calls the dispatch endpoint, which is delivered in seconds instead of queued, and the app reconciles that service's jobs against my schedule the way Terraform reconciles infrastructure. There is a human-in-the-loop mode where it opens a GitHub Issue and waits for approval, with a configurable timeout policy. And the whole thing has 466 tests, none of which call a model, because the model only writes prose — every decision about whether, when and how often is ordinary deterministic code."

Then stop talking. That pitch deliberately leaves four threads loose — the lateness, the reconciler, the approval timeout, and the test count. An interviewer will pull one. You have a full day's material behind every one of them, and an answer that arrives because they asked lands far better than the same answer volunteered.

The eight classic project questions

Question	The story you already own
"Tell me about a hard bug."	Incident 1 — three stacked causes.
"How do you decide what the LLM should and should not do?"	Your article, compressed to sixty seconds.
"How do you test AI features?"	466 tests, none needing a live model — because generation was isolated behind one function and everything else is deterministic.
"Tell me about a design tradeoff."	Server-rendered HTML with no frontend framework for a single-user local tool; or git-as-a-database instead of Postgres.
"How do you handle unreliable dependencies?"	Catch-up windows, retries that do not duplicate, failures recorded rather than swallowed — then the push trigger.
"A time you were wrong."	Assuming a scheduled run had published, when the post had actually been sent by hand from the UI. You checked the evidence and corrected yourself.
"How do you think about the user?"	"The product rule is that users must not depend on me. They take the code and run their own copy — so every integration has to be something they can sign up for themselves, and the README says exactly how. That constraint is why the cron service is external and configurable rather than a server I host."
"Something you shipped that a user actually uses."	You. Daily. Posting the article about the app, from the app.

Phrases worth having ready

On model boundaries

"I gave the model exactly one job. Everything about *whether* and *when* stayed in code I could test."

On reliability

"I designed for the trigger being late, because I measured that it usually was. Then I replaced the trigger."

On the reconciler

"It never says 'add a job'. It says: here is what should exist, here is what does, converge. Run it once or fifty times and you get the same thing."

On failure handling

"A failure that leaves no trace looks exactly like a quiet day. So every failure path writes a record, and the dashboard surfaces the ones nobody acted on."

On human oversight

"Silence is a decision. The question is only whether silence means publish or skip — so I made that configurable rather than assuming."

On boundaries between systems

"My text was fine and LinkedIn's parser disagreed about what a bracket meant. Every boundary between two systems is a chance for them to disagree about a character — that is the same bug as SQL injection, wearing different clothes."

On testing

"If your tests need a model, your decisions are in the wrong place."

On scope

"I did not build multi-platform support. One platform working reliably taught me more than three half-working ones."

Say none of these. "I used AI to build it" — they will ask what *you* decided. "Leveraged cutting-edge LLMs" — empty. "It is basically ChatGPT for LinkedIn" — you spent weeks on the part that is not the model; do not throw it away in one sentence. "Agentic AI" with no example behind it. "It just works" — you have twenty incidents proving otherwise, and saying so is a strength. And never claim RAG, fine-tuning, or multi-agent orchestration you did not build — one probing question and the whole interview turns.

THE LAST HOUR

1. Record the 90-second pitch. Watch it back. Re-record until you stop reaching for words.
2. Have someone — or an AI — ask you the eight questions above plus five random rows from Day 13's question map, out loud, in a random order, no notes.
3. Write the three questions that still made you hesitate. Those are the only things left to work on.

Deliverable: a recorded pitch you are willing to hear again, and a shortlist of your three remaining weak spots.

Day 15 onward, if you want it. The natural next steps are: adding a second platform to prove the abstraction actually abstracts; writing an evaluation harness that scores generated posts against a rubric; replacing the JSON files with SQLite to see what a real storage layer changes; and collapsing the five near-duplicate `_back()` helpers you deliberately left alone. Each is a week, and each adds a genuinely new entry to the incident bank — which, by now, you know is the part that interviews are actually made of.